

The Application of Genetic Algorithms to the Bin Packing Problem

ECM3412 - Nature-Inspired Computation

Candidate Number: 730002704

University of Exeter
Exeter, United Kingdom

Abstract—This report investigates the application of genetic algorithms to the Bin Packing Problem, a classic optimisation problem. A literature review of nature-inspired algorithms for the BPP is presented, followed by an implementation of a genetic algorithm to solve two instances of the problem.

Index Terms—genetic algorithms, bin-packing, optimisation, nature-inspired computation

GenAI was not used in the completion of this coursework.

I. INTRODUCTION

The Bin-Packing Problem (BPP) refers to the task of packing n items of varying weights into b bins such that the difference in weight between the heaviest and lightest bin is minimised [2].

Applications of the BPP range from table formatting to resource allocation [3].

In this report I investigate previous applications of nature-inspired algorithms to the BPP, implement a group of genetic algorithms to solve two instances of the BPP, and analyse the results obtained.

II. LITERATURE REVIEW

The Bin Packing Problem is an example of an NP-hard combinatorial problem [4]. As such, finding an optimal solution is infeasible for large instances, necessitating the use of heuristic approaches. In this literature review, I explore the use of nature-inspired algorithms for solving the BPP.

A. Genetic Algorithm Variants

To apply a standard genetic algorithm (GA) to the BPP, we need to choose a representation for candidate solutions. The most straightforward method is to use a list of integers, where the i -th integer represents the bin assigned to item i . A weakness of this representation is that it creates a *redundant search space*, where multiple genotypes map to the same phenotype. Consider that swapping all items between two bins does not change the solution, but results in a different genotype [1].

Falkenauer [1] proposes using a *grouping* representation, where the chromosome is a list of sets, each set representing the items in a bin. This eliminates redundancy but requires changing the operators used in the GA. Mutation involves either creating a new group, eliminating an existing group, or shuffling items amongst groups.

The performance of Falkenauer's grouping GA was evaluated against the MTP algorithm by Martello and Toth [1]. They found that the genetic algorithm outperforms MTP on large problem instances, whilst still approaching optimal solutions.

B. Particle Swarm Optimisation

Particle Swarm Optimisation (PSO) is an algorithm inspired by the flocking behaviour of birds. Each 'particle' in the swarm represents a candidate solution based on its position, and moves with some velocity determined by the algorithm. At each time step, each particle's velocity is updated to move it towards its own best-known position and the best-known position of the entire swarm. The weighting of these two influences is controlled by randomly generated coefficients [7].

Omar and Ramakrishnan [7] enhanced the standard PSO algorithm with standard concepts from evolutionary algorithms, such as mutation and crossover, to create an 'Evolutionary Particle Swarm Optimisation' (EPSO) algorithm. This algorithm was compared against a genetic algorithm (GA), simulated annealing (SA), and unified tabu search (UTS) on a set of benchmark BPP instances. They found that EPSO outperformed GA and SA, and produced comparable results to UTS.

It is likely that the improved performance of PSO over GA is due to its utilisation of randomness to better explore the search space without getting caught in local optima. This reduces the premature convergence often seen in GAs.

C. Ant Colony Optimisation

The Ant Colony Optimisation (ACO) algorithm works well for combinatorial problems by simulating the path-finding behaviour of ants. It uses a 'colony' of artificial ants that explore potential solutions. After an 'ant' has constructed a solution, it deposits an amount of pheromone along the path taken proportional to the quality of the solution. This pheromone guides subsequent ants towards promising areas of the solution space [5].

In order to solve the problem with ACO, we need to define two functions: the *pheromone trail definition* and *heuristic function*. The pheromone trail definition determines what is meant by the pheromone deposited by the ants. This is denoted $\tau(i, j)$ for the pheromone level between items i and j . This is defined to be the 'favourability' of having the two items in the same bin. The heuristic function, denoted $\eta(i)$, defines

the desirability of placing item i in the current bin. A simple heuristic function is to set $\eta(i)$ to be the weight of item i , so that heavier items are more likely to be placed in the current bin [6].

From these definitions the standard application of ACO can be used to solve the BPP. Each ant iteratively constructs a solution by selecting items to place in the current bin based on the pheromone levels and heuristic values. A parameter β is used to control how much influence the heuristic function has on the selection process. Once an ant has filled a bin, it moves on to the next bin and continues until all items have been placed. After all ants have constructed their solutions, the pheromone levels are updated based on the quality of the solutions found [6].

Levine and Ducatelle [6] found optimal parameters through experimentation. They found best results when the number of ants was equal to the number of items, whilst the rest of the parameters appeared not to be linked to features of the problem instance.

A weakness of this approach is its reliance on parameter tuning. This requires running the algorithm multiple times with different parameter settings to find the best configuration, which can be time-consuming. Since these parameters are based on specific problem instances, they will also not generalise well to other instances of the BPP.

D. Other Nature-Inspired Approaches

Simulated Annealing (SA) mimics the process of a solid slowly cooling until it freezes in a low-energy state. This is done by iteratively generating ‘neighbouring’ solutions to the current candidate. This solution is always accepted if it is better than the current solution, but may sometimes be accepted even if it is worse, with probability $e^{-\frac{\delta}{T}}$, where δ is the difference in fitness between the two solutions, and T is the current ‘temperature’ which decreases over time [8]. Key parameters of the algorithm are the initial temperature, cooling rate, and stopping temperature.

Rao and Iyengar [9] applied simulated annealing to the same BPP variant that will be implemented later in this report. They found that, compared to the classic Largest Piece First (LPF), Smallest Piece First (SPF), and First Fit Decreasing (FFD) heuristics, SA produced better results with lower variance and more stably across instances.

Tabu Search (TS) also iterates through neighbouring solutions, but maintains a *tabu list* of recently visited solutions to avoid cycling back to them. The algorithm selects the best non-tabu neighbour at each iteration, allowing it to escape local optima [10].

Buljubašić and Vasquez [11] applied TS to the Bin Packing Problem. They initialised the algorithm with a complete solution generated by the First Fit heuristic on a random permutation of the items. Two solutions are considered neighbours if they differ by a single addition or removal of an item from a bin. This algorithm was compared to a GA and the BPP-specific algorithm ‘Perturbation-SAWMB’. They found that whilst all three algorithms could find the optimal solution

for simpler instances, on more complex instances TS found significantly better solutions than the other two algorithms. Whilst Perturbation-SAWMB was slightly faster than TS, both were an order of magnitude slower than the GA.

A weakness of the TS approach is that it requires careful selection of the neighbourhood structure to be effective. A poorly chosen neighbourhood can lead to slow convergence or getting stuck in local optima. A method of generating initial solutions is also required. In general, TS also has a high number of tunable parameters that need to be optimised for each problem instance.

E. Critical Analysis

A common downside of all the genetic approaches is their reliance on parameter tuning. Whilst some, like PSO, have fewer parameters to tune, some like ACO and TS have many parameters that need to be optimised for each problem instance. This can lead to ineffective performance if the parameters are not well-chosen.

In all the literature, the genetic approaches outperform traditional heuristics like FFD and LPF. They are often able to find better solutions with lower variance across multiple runs. This is likely due to the nature of the problem having a large search space with many local optima, which nature-inspired algorithms are well-suited to explore.

A study which directly compares the different nature-inspired algorithms on a range of the same problem instances would be useful to determine which approach is most effective for the BPP, and which conditions favour each approach.

III. DESCRIPTION OF RESULTS

A. Experimental Setup

The parameters for the two Bin Packing Problems are given in Table I. Table II gives the four configurations of parameters used in the genetic algorithm experiments.

Each configuration was run five times with different random seeds for 1,000 fitness evaluations. The best fitness, mean fitness, and standard deviation of fitness across the five runs were recorded, and the average fitness at each iteration was recorded for plotting.

TABLE I: Description of Bin Packing Problems. The weight function $w(i)$ gives the weight of item i , where $i \in [1, 500]$.

Problem	Number of Bins	Weight Function
BPP1	10	$w(i) = i$
BPP2	50	$w(i) = \frac{i^2}{2}$

TABLE II: The parameter values to be used.

Parameter	Conf. 1	Conf. 2	Conf. 3	Conf. 4
Population Size p	100	100	100	100
Mutation Rate p_m	0.01	0.05	0.01	0.05
Tournament Size t	3	3	7	7

B. Experimental Results

1) *BPP1 Results (10 bins, 500 items)*: The results for BPP1 are summarised in Table III, and the average fitness over iterations is shown in Figure 3.

TABLE III: BPP1 Results Summary

p_m	t	Best Fitness	Mean Fitness	Std. Dev
0.01	3	7.84×10^{-3}	7.32×10^{-3}	2.03×10^{-4}
0.05	3	7.70×10^{-3}	6.76×10^{-3}	3.87×10^{-4}
0.01	7	7.90×10^{-3}	7.59×10^{-3}	1.25×10^{-4}
0.05	7	7.73×10^{-3}	7.10×10^{-3}	2.75×10^{-4}

2) *BPP2 Results (50 bins, 500 items)*: The results for BPP2 are summarised in Table IV, and the average fitness over iterations is shown in Figure 4.

TABLE IV: BPP2 Results Summary

p_m	t	Best Fitness	Mean Fitness	Std Dev
0.01	3	1.96×10^{-4}	1.69×10^{-4}	1.14×10^{-5}
0.05	3	1.75×10^{-4}	1.34×10^{-4}	1.20×10^{-5}
0.01	7	2.01×10^{-4}	1.83×10^{-4}	1.19×10^{-5}
0.05	7	1.83×10^{-4}	1.49×10^{-4}	1.30×10^{-5}

IV. DISCUSSION AND FURTHER WORK

A. *Question 1: Which combination of parameters produces the best results?*

In both instances, the configuration with $p_m = 0.01$ and $t = 7$ led to the highest mean and best fitness values, as can be seen in Tables III and IV. The standard deviation for this configuration was also the lowest for DPP1, and second lowest for DPP2, indicating more consistent performance.

B. *Question 2: What do you think is the reason for your findings in Question 1?*

A lower mutation rate helps to preserve beneficial traits in the population. A larger tournament size increases selection pressure, favouring fitter individuals. Decreasing the mutation rate too far would limit how much the population could explore the search space over the limited iterations, while increasing the tournament size too much would lead to premature convergence.

C. *Question 3: How do each of the parameter settings influence the performance of the algorithm?*

By comparing the groups of graphs for each instance given in Figures 3 and 4, we can see the effects of changing each parameter:

- $p_m = 0.01$ always leads to a higher mean fitness than $p_m = 0.05$. This is likely because a lower mutation rate helps to preserve beneficial traits in the population.
- At $t = 7$ the population converges more quickly than at $t = 3$. Note how much steeper the initial fitness increase is in 3 (c) compared to (a), and similarly in 4. This is because a larger tournament size increases selection

pressure, favouring fitter individuals and reducing the survival of less fit individuals.

- In the first instance, BPP1, setting $t = 7$ leads to a lower (by around 40%) standard deviation than $t = 3$, indicating more consistent performance. In BPP2 there is a slight increase. This may be because the larger search space of BPP2 may lead to premature convergence when using a larger tournament size.

D. *Question 4: Do you think that one of the algorithms in your literature review might have provided better results? Explain your answer.*

From the literature review, it seems that Ant Colony Optimisation (ACO) and Evolutionary Particle Swarm Optimisation (EPSO) have both been shown to outperform genetic algorithms on the Bin Packing Problem [6], [7]. These algorithms are better at avoiding local optima due to their increased diversity in the population.

ACO uses pheromone trails to guide the search towards promising areas of the solution space, while EPSO uses velocity updates. Both of these mechanisms maintain the diversity of the population better than standard genetic algorithms, which can suffer from premature convergence due to selection pressure and loss of genetic diversity.

E. Further Experiments

My first experiment was into the role of population size on the performance of the algorithm. Using BPP2 with $p_m = 0.01$ and $t = 7$, I ran the algorithm with population sizes of 10, 50, 100, 400, and 800 for three trials each. The results are shown in Figure 1. The number of iterations was fixed, so the number of fitness evaluations increased with population size.

I found that increasing the population size tended to decrease the average fitness. This could be due to the search space being able to be explored with only a few individuals, and so a small population can more efficiently find good solutions. Over higher iterations and with different parameters, a large population could be more beneficial to maintain diversity.

The second experiment varied the mutation rate more widely, using values of 0.0001, 0.001, 0.01, 0.1, and 1.0, with $p = 100$ and $t = 7$ on BPP2. Each configuration was run three times. The results are shown in Figure 2.

The best performance was achieved with $p_m = 0.001$. The performance of $p_m = 0.0001$ was slightly lower, likely because the mutation rate was too low to explore the search space effectively. Higher mutation rates led to significantly worse performance, as excessive mutation disrupts beneficial traits in the population. At $p_m = 1.0$, the algorithm cannot retain any beneficial traits, leading to random search behaviour.

F. Future Work

It would be interesting to explore the effects of modifying the parameters over the course of the algorithm's execution, such as using a higher mutation rate in early generations to encourage exploration, and lowering it later to refine solutions.

A different encoding with less redundancy could be investigated, for instance the grouping representation proposed

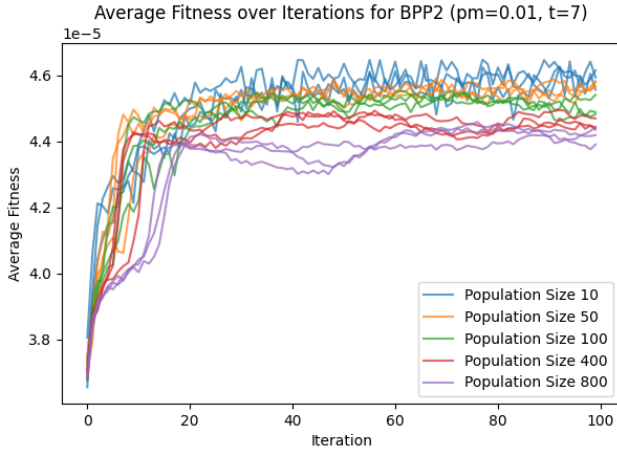


Fig. 1: Effect of Population Size on Average Fitness over Iterations for BPP2 with $p_m = 0.01$ and $t = 7$.

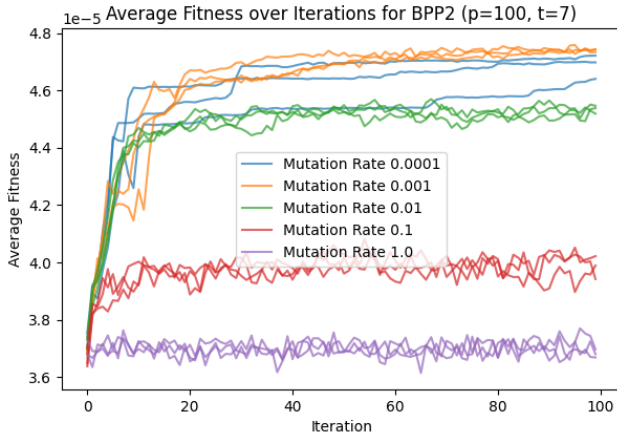


Fig. 2: Effect of Mutation Rate on Average Fitness over Iterations for BPP2 with $p = 100$ and $t = 7$.

by Falkenauer [1]. This would require custom operators and a more complex implementation to deal with the encoding scheme, but could potentially yield better results.

V. CONCLUSION

In this report I have implemented a GA to solve two instances of the BPP. The results indicate that a lower mutation rate and larger tournament size improve the algorithm's performance.

From the results, we can see that the instances converge more quickly on BPP1 and on a fitter solution than BPP2, likely due to the smaller search space. The best configurations also result in a low standard deviation, indicating consistent performance.

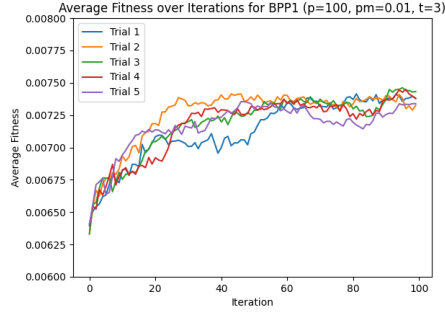
My further experiments into population size and mutation rate show that smaller populations and lower mutation rates

tend to yield better results, although there is a balance to be struck to maintain diversity.

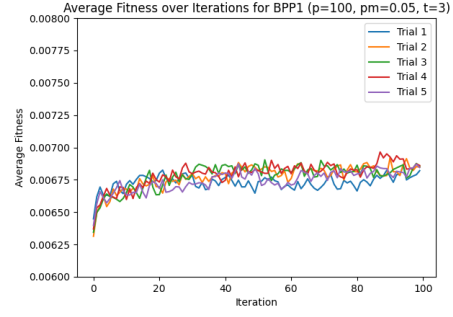
REFERENCES

- [1] E. Falkenauer, "A hybrid grouping genetic algorithm for bin packing," *J Heuristics*, vol. 2, no. 1, pp. 5–30, 1996, doi: 10.1007/BF00226291.
- [2] M. Delorme, M. Iori, and S. Martello, "BPPLIB: a library for bin packing and cutting stock problems," *Optim Lett*, vol. 12, no. 2, pp. 235–250, Mar. 2018, doi: 10.1007/s11590-017-1192-z.
- [3] D. S. Johnson, "Fast algorithms for bin packing," *Journal of Computer and System Sciences*, vol. 8, no. 3, pp. 272–314, Jun. 1974, doi: 10.1016/S0022-0000(74)80026-7.
- [4] H. Dyckhoff, "A typology of cutting and packing problems," *European Journal of Operational Research*, vol. 44, no. 2, pp. 145–159, Jan. 1990, doi: 10.1016/0377-2217(90)90350-K.
- [5] M. Dorigo, V. Maniezzo, and A. Coloni, "Ant system: optimization by a colony of cooperating agents," *IEEE Trans. Syst., Man, Cybern. B*, vol. 26, no. 1, pp. 29–41, Feb. 1996, doi: 10.1109/3477.484436.
- [6] J. Levine and F. Ducatelle, "Ant colony optimization and local search for bin packing and cutting stock problems," *Journal of the Operational Research Society*, vol. 55, no. 7, pp. 705–716, Jul. 2004, doi: 10.1057/palgrave.jors.2601771.
- [7] M. K. Omar and K. Ramakrishnan, "Solving non-oriented two dimensional bin packing problem using evolutionary particle swarm optimisation," *International Journal of Production Research*, vol. 51, no. 20, pp. 6002–6016, Oct. 2013, doi: 10.1080/00207543.2013.791754.
- [8] D. Bertsimas and J. Tsitsiklis, "Simulated Annealing," *Statist. Sci.*, vol. 8, no. 1, Feb. 1993, doi: 10.1214/ss/1177011077.
- [9] R. L. Rao and S. S. Iyengar, "Bin-packing by simulated annealing," *Computers & Mathematics with Applications*, vol. 27, no. 5, pp. 71–82, Mar. 1994, doi: 10.1016/0898-1221(94)90077-9.
- [10] F. Glover and M. Laguna, *Tabu Search*. Boston, MA: Springer US, 1997, doi: 10.1007/978-1-4615-6089-0.
- [11] M. Buljubašić and M. Vasez, "Consistent neighborhood search for one-dimensional bin packing and two-dimensional vector packing," *Computers & Operations Research*, vol. 76, pp. 12–21, Dec. 2016, doi: 10.1016/j.cor.2016.06.009.

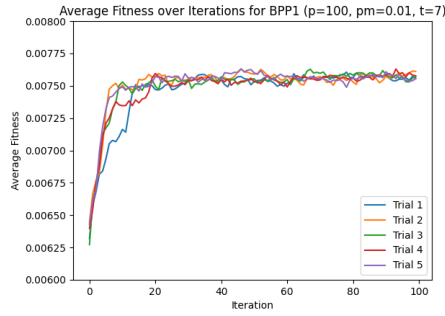
Fig. 3: BPP1 Results. Note that the axis scales are consistent across the subfigures.



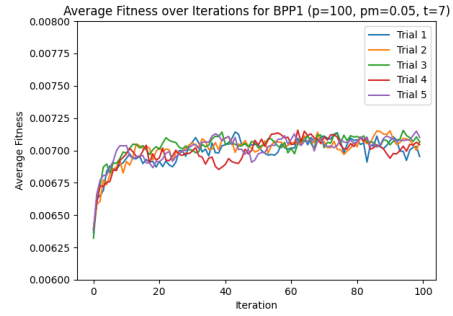
(a) $p_m = 0.01$, $t = 3$



(b) $p_m = 0.05$, $t = 3$

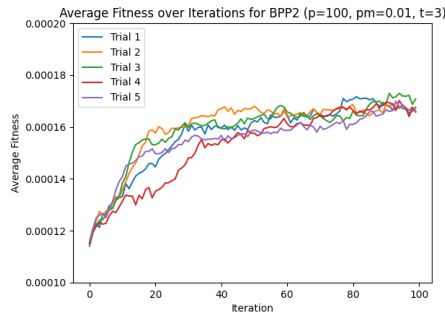


(c) $p_m = 0.01$, $t = 7$

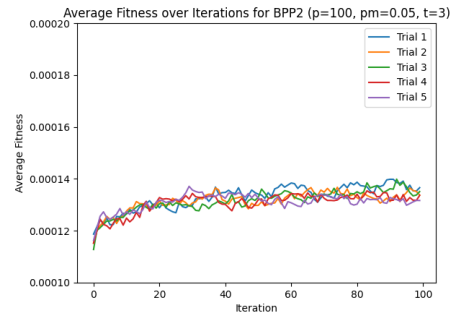


(d) $p_m = 0.05$, $t = 7$

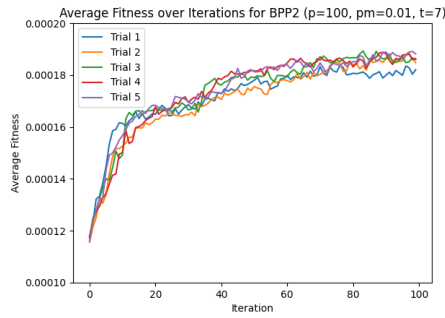
Fig. 4: BPP2 Results. Note that the axis scales differ from those in Figure 3 but are consistent across the subfigures here.



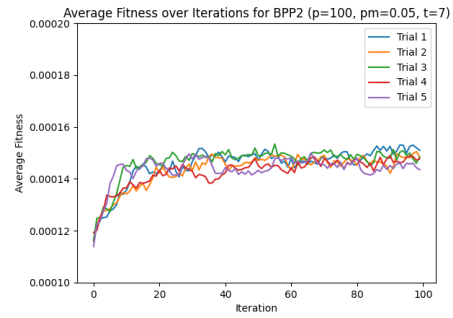
(a) $p_m = 0.01$, $t = 3$



(b) $p_m = 0.05$, $t = 3$



(c) $p_m = 0.01$, $t = 7$



(d) $p_m = 0.05$, $t = 7$